

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

РОЗРАХУНКОВА РОБОТА

з дисципліни “Об’єктно-орієнтоване програмування”

на тему:

«Розробка програмного комплексу системи «Електронна книга рецептів»

Анотація

За допомогою технології Windows Forms реалізовано програму «Електронна книга рецептів». Програма написана мовою C++. Мета розрахункової роботи є продемонструвати, на прикладі даної програми, можливості розробки інтерфейсу користувача у Windows Forms.

Зміст

РОЗДІЛ 1. Варіантний огляд та аналіз сучасних методів та засобів проектування програмного забезпечення комп'ютерних систем управління.....	5
1. 1 Аналіз об'єкту автоматизації.....	5
1. 2 Класифікація об'єктно-орієнтованих мов програмування.....	7
1. 3 Огляд та аналіз сучасних технологій та засобів проектування програмного забезпечення КСУ	12
1. 4 Універсальна мова проектування UML	13
1. 5 Уточнена постановка задачі на розробку програмного забезпечення...	14
РОЗДІЛ 2 Проектування програмного забезпечення системи. Формулювання вимог до системи	16
2. 1 Етап формування вимог до системи.....	16
2. 2 Розробка UML діаграми варіантів використання	18
2. 3. Розробка UML діаграм поведінки системи.....	20
2. 3. 1 UML діаграма послідовності	20
2. 3. 2 UML діаграма діяльності	24
2. 4. Розробка графічного інтерфейсу програмних засобів комп'ютерної системи	25
РОЗДІЛ 3Розробка програмного забезпечення системи	28
3. 1 Розробка UML діаграм класів.....	28
3. 2 Опис класів програмного комплексу	28
3. 3 Розробка структури бази даних	29
РОЗДІЛ 4 Тестування програмного забезпечення	31
4. 1Розробка тестів	31
РОЗДІЛ 5 Розробка документів на супроводження програмного забезпечення.....	33
5. 1 Інструкція користувачеві	33
Висновок	36
Додаток	38

ВСТУП

Об'єктно-орієнтоване програмування (ООП) – це еволюційний крок, який впливає із розвитку програмування. ООП дає нам можливість відчувати себе не тільки програмістом, а й архітектором, проектуючи структуру програми, створюючи красиві форми.

Ціль розрахункової роботи продемонструвати основи роботи технології Windows Forms на прикладі програми «Електронна книга рецептів».

В сучасному світі людство оперує безмежною кількістю інформації, яку зберігати та сортувати без допоміжних засобів просто неможливо. Тому актуальність програм-каталогізаторів сьогодні дуже висока. Вони допомагають упорядковувати, знаходити та порівнювати певну структуру даних.

Задача розробки кулінарної книги також вимагає створення каталогізатора кулінарних рецептів, щоб можна було легко, а головне, швидко знайти потрібний рецепт, створити новий, перемістити будь-який рецепт до будь-якої категорії тощо.

РОЗДІЛ 1. Варіантний огляд та аналіз сучасних методів та засобів проектування програмного забезпечення комп'ютерних систем управління

1. 1. Аналіз об'єкту автоматизації

Об'єктом автоматизації є книга рецептів.

Книга рецептів – видання, що вміщує в собі колекцію рецептів приготування їжі. Сучасні версії можуть також вміщувати барвисті ілюстрації та рекомендації щодо купівлі якісних інгредієнтів або їх заміни. Кулінарна книга також може охоплювати широкий діапазон теми, в тому числі: історичні довідки про традиційні кухні різних держав, методи приготування їжі на щодень чи для урочистих подій, рецепти та коментарі від відомих шеф-кухарів, діячів культури та мистецтва, а також культурний коментар.

Електронний довідник – навчальні, наукові, інформаційні, довідкові матеріали та засоби, розроблені в електронній формі і представлені на носіях будь-якого типу, або розміщені у комп'ютерних мережах, які відтворюються за допомогою електронних цифрових технічних засобів і необхідні для ефективної організації навчально-виховного процесу, в частині, що стосується його наповнення якісними навчально-методичними матеріалами. ЕОР є важливим інструментом навчально-виховного процесу, має навчально-методичне призначення та використовується для забезпечення навчальної діяльності вихованців, учнів, студентів і вважається одним з головних елементів інформаційно-освітнього середовища. Метою створення ЕОР є змістове наповнення освітнього простору, забезпечення рівного доступу учасників навчально-виховного процесу до якісних навчальних та методичних матеріалів незалежно від місця їх проживання та форми навчання, створених на основі інформаційно-комунікаційних технологій.

Аналогічні системи.

Для аналізу аналогічної системи використаємо систему «Кулінарна книга». Для розробки програми було використано середовище Visual Studio

2019. Програма містить два меню (головне та контекстне), а також панель управління. Можна звертатися до пунктів меню, або кнопок панелі управління у будь-якому порядку. Вибір відповідних пунктів меню здійснюється мишкою. Також є можливість користуватися гарячими клавішами. Про відповідність гарячих клавіш можна дізнатися із головного меню, кожний пункт якого містить інформацію про відповідну йому комбінацію клавіш. Програма дозволяє створювати необмежену кількість категорій і рецептів. Є лише одна умова: для користування програмою необхідно мати хоча б одну категорію, тому програма не дозволить користувачеві видалити єдину категорію. Є можливість виконувати багато дій над рецептами і категоріями. До них відносяться: створення, редагування, видалення, копіювання, переміщення, перейменування, пошук, завантаження з текстового файлу, збереження в текстовий файл.

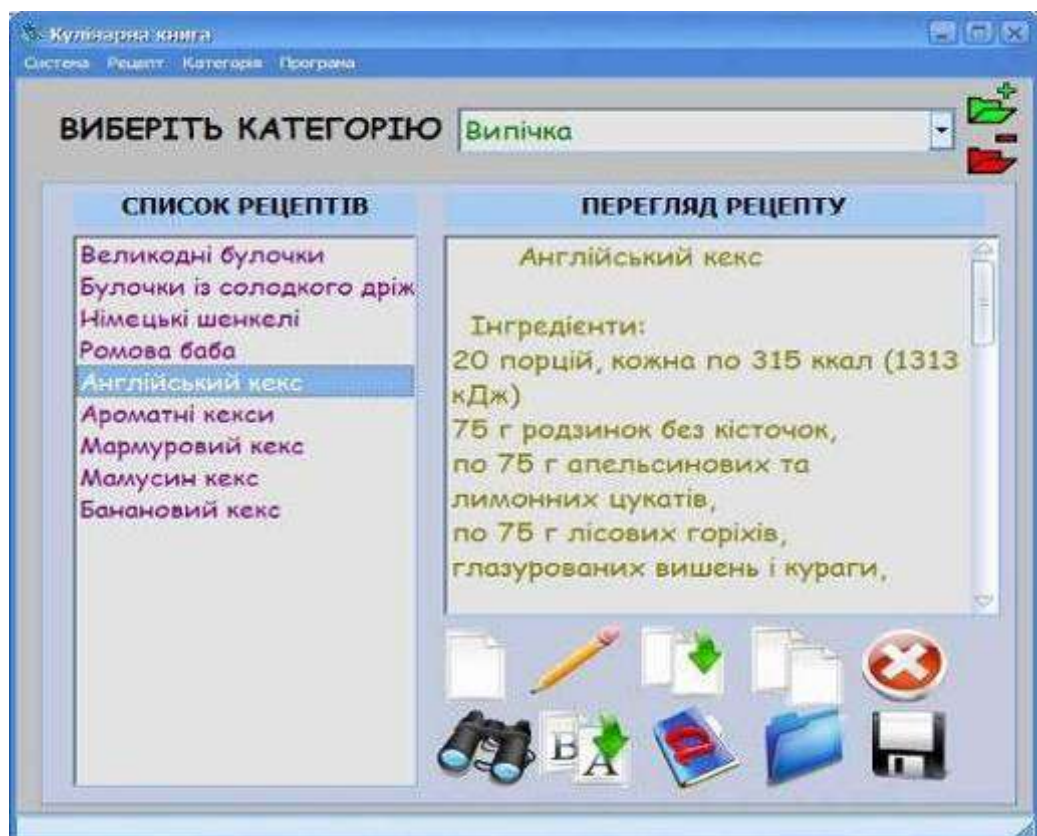


Рис. 1. Аналогічна система.

Ця система має такі переваги і недоліки:

Переваги – простота, надійність, дружній інтерфейс, працює у віконному режимі, розвита графіка.

Недоліки – великий обсяг необхідної оперативної пам'яті (близько 8 МБ), також немає візуалізації рецептів та можливості роздрукувати той чи інший рецепт.

1. 2. Класифікація об'єктно-орієнтованих мов програмування

Класифікацій мов програмування існує багато, але наукової теорії поки що немає. Три основні класифікації склалися історично:

1. За функціональною силою:

- універсальні мови (в них можна промодельовати, умовно кажучи, будь-який алгоритм) ;
- спеціалізовані мови (орієнтовані на певні класи задач).

2. За предметною орієнтацією:

Кожна мова програмування виникла в процесі розв'язання певного класу задач, наприклад, мови програмування для розв'язання задач символічної обробки (Lisp, Cobol) і т. п.

3. За рівнем абстракції:

- мови низького рівня (машинно-залежні) – Assembler і т. п. ;
- мови високого рівня (орієнтовані на користувача (людину) до певної міри) – Pascal, C, Fortran і т. п. ;

Мови програмування низького рівня – орієнтовані на конкретний тип процесора і враховують його особливості.

Переваги:

- З допомогою мов низького рівня створюються ефективні і компактні програми, оскільки розробник отримує доступ до всіх можливостей процесора. Недоліки:
- Програміст, що працює з мовами низького рівня, має бути високої кваліфікації, добре розуміти будову комп'ютера.
- Результуюча програма не може бути перенесена на комп'ютер з іншим типом процесора.

Асемблер

Мова Асемблера поєднує в собі достоїнства мови машинних команд і деякі риси мов високого рівня. Асемблер забезпечує можливість застосування символічних імен у вихідній програмі й рятує програміста від стомлюючої праці (неминучого при програмуванні мовою машинних команд) по розподілі пам'яті комп'ютера для команд, змінних і констант. Асемблер дозволяє також гнучко й повно використати технічні можливості комп'ютера, як і мова машинних команд. Транслятор вихідних програм в Асемблері простіше транслятора, що вимагається для мови програмування високого рівня. На Асемблері можна написати настільки ж ефективну за розміром й часом виконання програму, як і програму мовою машинних команд. Ця перевага відсутня в мов високого рівня. Цю мову часто застосовують для програмування систем реального часу, технологічними процесами й обладнанням, забезпечення роботи інформаційно-вимірювальних комплексів. До таких систем звичайно пред'являються високі вимоги за обсягом займаної машинної пам'яті. Часто мова Асемблера доповнюється засобами формування макрокоманд, кожна з яких еквівалентна цілій групі машинних команд. Таку мову називають мовою макроасемблера.

Мови програмування високого рівня.

Мови програмування високого рівня дозволяють писати програми в формі, більш наближеній до звичайної мови. Програму, написану мовою високого рівня, можна більш легко читати і модифікувати, і вони значно полегшують роботу програміста порівняно з написанням машинного коду. Для перекладу програм, написаних мовою високого рівня, в машинні коди, повинні існувати спеціальні програми. Такі програми називаються трансляторами. Важливою особливістю мов високого рівня є їх відносна незалежність від машини. Це означає, що правила запису програм не залежать, або мало залежать від особливостей конкретної машини. Тоді для перенесення програми на іншу машину не обов'язково переписувати заново, достатньо лише відтранслювати її в коди, специфічні для цієї машини. В

крайньому разі, зміни в програмі повинні бути мінімальними.

C++

Мова C++ з'явився на початку 80-х років.

Створений Б'єрном Страуструпом з первісною метою позбавити себе й своїх друзів від програмування на асемблері, Сі або різних інших мовах високого рівня.

Очевидно, що найбільше C++ запозичив з мови Сі, а також з безпосереднього його попередника мови BCPL.

Ці запозичення забезпечили C++ потужними засобами низького рівня, що дозволяють вирішувати складні завдання системного програмування. Але що в першу чергу відрізняє C++ від Сі – це різний ступінь уваги до типів і структур даних. Це пов'язано з появою понять класу, похідного класу й віртуальної функції, перейнятих у свою чергу з мови Симула 67.

Це дає в C++ більш ефективні можливості для контролю типів і забезпечує модульність програми.

На думку автора мови, розходження між ідеологією С й C++ полягає приблизно в наступному: програма на С відображає “спосіб мислення” процесора, а C++ – спосіб мислення програміста. Відповідаючи вимогам сучасного програмування, C++ наголошує на розробці нових типів даних найбільше повно відповідним концепціям обраної області знань. Клас є ключовим поняттям C++. Опис класу містить опис даних, що вимагаються для подання об'єктів цього типу й набір операцій для роботи з подібними об'єктами.

На відміну від традиційних структур С й Паскаля, членами класу є не тільки дані, але й функції. Функції – члени класу мають привілейований доступ до даних усередині об'єктів цього класу й забезпечують інтерфейс між цими об'єктами й іншою програмою. При подальшій роботі зовсім не обов'язково пам'ятати про внутрішню структуру класу й механізм роботи вбудованих функцій. У цьому змісті клас подібний до електричного приладу – мало хто знає про його пристрій, але всі знають, як ним користуватися.

Мова C++ є засобом об'єктного програмування, новітньої методики проектування й реалізації програм, що у поточному десятилітті, швидше за все, замінить традиційне процедурне програмування.

Головною метою творця мови доктора Б'єрна Страустрапа було оснащення мови C++ конструкціями, що дозволяють збільшити продуктивність праці програмістів і полегшити процес оволодіння великими програмними продуктами.

Абстракція, реалізація, спадкування й поліморфізм є необхідними властивостями якими володіє мова C++, завдяки чому вона не тільки універсальна, як і мова C, але і є об'єктною мовою.

Cі

Співробітник фірми BellLabs Денис Рітчі створив мову Cі в 1972 році під час спільної роботи з Кеном Томпсоном, як інструментальний засіб для реалізації операційної системи Unix, однак популярність цієї мови швидко переросла рамки конкретної операційної системи й конкретних завдань системного програмування. У цей час будь-яка інструментальна й операційна система не може вважатися повною якщо в її состав не входить компілятор мови Cі.

Рітчі не видумував Cі просто з голови – прообразом була мова Би розроблена Томпсоном. Мова програмування Cі була розроблена як інструмент для програмістів-практиків. Відповідно до цим головною метою його автора було створення зручної й корисної мови.

Cі є знаряддям системного програміста й дозволяє глибоко влазити в самі тонкі механізми обробки інформації на ЕОМ. Хоча мова жадає від програміста високої дисципліни, вона не строга у формальних претензіях і допускає короткі формулювання.

Cі – сучасна мова. Він містить у собі ті керуючі конструкції, які рекомендовані теорією й практикою програмування. Його структура спонукує програміста використовувати у своїй роботі спадне проектування, структурне програмування й покрокову розробку модулів.

Cі – ефективна мова. Її структура дозволяє щонайкраще використати можливості сучасних ПЕОМ. Програмування на цій мові відрізняється компактністю й швидкістю виконання.

Cі – мобільна мова.

Cі – потужна й гнучка мова. Більша частина операційної системи Unix, компілятори й інтерпретатори мов Фортран, Паскаль, Лісп, і Бейсик написані саме з його допомогою.

Cі – зручна мова. Він досить структурований, щоб підтримувати гарний стиль програмування й разом з тим не зв'язаний твердими обмеженнями.

Java

Мова Java зародилась як частина проекту створення передового програмного забезпечення (ПО) для різних побутових приладів. Реалізація проекту була почата мовою C++, але незабаром виник ряд проблем, найкращим засобом боротьби з якими була зміна самого інструмента – мови програмування. Стало очевидним, що необхідно платформи-незалежна мова програмування, що дозволяє створювати програми, які не доводилося б компілювати окремо для кожної архітектури й можна було б використати на різних процесорах під різними операційними системами.

Три ключових елементи об'єдналися в технології мови Java:

Java надає для широкого використання свої аплети (applets) – невеликі, надійні, динамічні, що не залежать від платформи активні мережні додатки, що вбудовують у сторінки Web. Аплети Java можуть налаштовуватися й поширюватися споживачам з такою же легкістю, як будь-які документи HTML.

Java вивільняє міць об'єктно-орієнтованої розробки додатків, сполучаючи простий і знайомий синтаксис із надійним й зручним в роботі середовищем розробки.

Це дозволяє широкому колу програмістів швидко створювати нові програми й нові аплети.

Java надає програмістові багатий набір класів об'єктів для ясного

абстрагування багатьох системних функцій, використовуваних при роботі з вікнами, мережею й для вводу-виводу. Ключова риса цих класів полягає в тім, що вони забезпечують створення незалежних від використовуваної платформи абстракцій для широкого спектра системних інтерфейсів.

1. 3. Огляд та аналіз сучасних технологій та засобів проектування програмного забезпечення КСУ

Комп'ютерна система управління (КСУ) – автоматизована система, що ґрунтується на комплексному використанні технічних, математичних, інформаційних та організаційних засобів для управління складними технічними й економічними об'єктами. КСУ – це сукупність керованого об'єкта й автоматичних вимірювальних та керуючих пристроїв, у якій частину функцій виконує людина.

Створені за тридцятилітню історію впровадження ЕОМ у сферу управлінської діяльності численні КСУ різняться призначенням, проблемною орієнтацією, місцем застосування, автоматизованими функціями і т. ін. З метою підвищення ефективності витрат на розвиток діючих систем та проектування нових, усунення паралелізму і дублювання в проведенні наукових досліджень і проектно-конструкторських робіт, створення типових проектних рішень і типових КСУ зроблено їх класифікацію.

КСУ дає змогу розв'язувати задачі перспективного та оперативного планування виробництва, оперативного розподілу завантаження обладнання, оптимального розподілу обладнання та використання ресурсів і інше. АСК належить до класу людино-машинних систем і складається з функціональної і забезпечувальної частин.

Функціональна частина КСУ включає систему моделей планово-економічних і управлінських задач, забезпечувальна частина – інформаційну і технічну бази, математичне забезпечення, економіко-організаційну базу та інше.

Спеціальне математичне забезпечення включає пакети прикладних програм, що здійснюють організацію й обробку даних з метою реалізації

необхідних функцій управління в рамках певних економіко-математичних та організаційних моделей. Програмне забезпечення КСУ (ПЗ) містить сукупність програм на носіях, даних і програмних документів, яка призначена для відлагодження, функціонування й перевірки роботоздатності КСУ.

1. 4. Універсальна мова проектування UML

UML (англ. UnifiedModelingLanguage) – уніфікована мова моделювання, використовується у парадигмі об'єктно-орієнтованого програмування. Є невід'ємною частиною уніфікованого процесу розробки програмного забезпечення. UML є мовою широкого профілю, це відкритий стандарт, що використовує графічні позначення для створення абстрактної моделі системи, названої UML-моделлю. UML був створений для визначення, візуалізації, проектування й документування в основному програмних систем. UML не є мовою програмування, але в засобах виконання UML-моделей як інтерпретованого коду можлива кодогенерація. Перша версія (1. 0) UML вийшла 13 січня 1997, вона була створена за запитом ObjectManagementGroup (OMG) – організації, відповідальної за прийняття стандартів в галузі об'єктних технологій і баз даних. Після обговорення, у вересні 1997 року, версія 1. 1 UML була представлена на голосування в OMG. Розробку UML підтримали і вже тоді використовували як стандарт такі гранди ринку інформаційних технологій, як Microsoft, IBM, Hewlett-Packard, Oracle, DEC, Sybase, Logic Works й інші.

UML може бути застосовано на всіх етапах життєвого циклу аналізу бізнес-систем і розробки прикладних програм. Різні види діаграм які підтримуються UML, і найбагатший набір можливостей представлення певних аспектів системи робить UML універсальним засобом опису як програмних, так і ділових систем. Діаграми дають можливість представити систему (як ділову, так і програмну) у такому вигляді, щоб її можна було легко перевести в програмний код.

Основною причиною використання мови UML є спілкування

розробників між собою.

Крім того, UML спеціально створювалася для оптимізації процесу розробки програмних систем, що дозволяє збільшити ефективність їх реалізації у кілька разів і помітно поліпшити якість кінцевого продукту.

UML прекрасно зарекомендувала себе в багатьох успішних програмних проектах. Засоби автоматичної генерації кодів дозволяють перетворювати моделі мовою UML у вихідний код об'єктно-орієнтованих мов програмування, що ще більш прискорює процес розробки.

Практично усі CASE-засоби (програми автоматизації процесу аналізу і проектування) мають підтримку UML. Моделі розроблені в UML, дозволяють значно спростити процес кодування і направити зусилля програмістів безпосередньо на реалізацію системи.

Діаграми підвищують супроводжуваність проекту і полегшують розробку документації.

1. 5. Уточнена постановка задачі на розробку програмного забезпечення

На основі виконаного аналізу та огляду літературних джерел можна сформулювати постановку задачі: розробити програмне забезпечення інформаційної системи « Електронна книга рецептів », основними функціями якої є: можливість пошуку по параметрах, можливість перегляду всіх рецептів програми.

Розробити зручний графічний інтерфейс для роботи з інформацією. Основні дії та взаємодія між користувачем та системою повинні супроводжуватися відповідними повідомленнями для користувача

Створити навігаційні клавiші для можливості швидкого та зручного отримання доступу до потрібної функції в системі.

Програма «Електронна книга рецептів » повинна бути універсальною, такою, щоб підходила як для домашнього користування, так і для установ громадського харчування.

Для розробки подібного продукту потрібно орієнтуватися на те, що користувач цієї програми може мати початкові навички роботи з

персональним комп'ютером, тому інтерфейс програми повинен бути якомога простішим, з підказками, з контекстним меню, з приємним дизайном, щоб зробити користування цією програмою простим і приємним.

РОЗДІЛ 2 Проектування програмного забезпечення системи. Формулювання вимог до системи

2. 1. Етап формування вимог до системи Завдання системи

Програмний продукт оперує даними про рецепти.

Завдання системи

Система повинна застосовуватися для отримання довідкової інформації про приготування тої чи іншої страви. Призначається для полегшення приготування різних страв.

Загальна характеристика системи

Система повинна виконувати основні функції:

- Можливість виведення всіх рецептів на екран.
- Можливість виведення детальної інформації про вибраний рецепт.
- Можливість пошуку за певними критеріями.
- Можливість зміни даних (додавання, вилучення, редагування).
- Дані про рецепти зберігаються у відповідних текстових файлах.
- Можливість задання під кожен рецепт відповідного зображення.
- Можливість роздруку рецептів.

Умови роботи

Комп'ютерна програма для пошуку потрібного рецепту. Операції з даними про рецепт можуть розширюватись.

Не функціональні вимоги

- Для звичайної роботи програми достатньо комп'ютера з монітором, клавіатурою та мишкою.
- Програма повинна без затримок здійснювати пошук.
- Простий і зрозумілий інтерфейс

Користувачі програми

Програма є універсальною, такою, що підходить як для домашнього користування, так і для установ громадського харчування.

Якість

Порівняно з іншими програмними продуктами система пропонує правильну інформацію про рецепти, має можливість швидкого пошуку. Збоїв програми при тестуванні не виявлено.

Продуктивність

Продуктивність системи залежить від швидкості пошуку, який не повинен тривати довше 200-350мсек., швидкості внесенню змін (100мсек).

У випадку використання фіскального принтера на продуктивність системи впливає драйвер пристрою. Для такого випадку потрібно реалізувати драйвер обміну даних системи з принтером який одночасно з запитом видає результат виконання операції. Допускається затримка на виконання операції фіскальним пристроєм не більше ніж 1сек. Збільшення затримки на необхідну кількість часу береться з протоколу самого принтера. Результат операції принтера виводиться на екран, або цифрове табло для користувача тоді перевірити і вивести на екран одну з причин несправностей а саме:

- Занижена, або завищена напруга
- Внутрішня помилка
- Немає зв'язку з принтером
- Принтер не готовий для виконання операцій

Відношення з іншими програмами.

Система працює на всіх ОС Microsoft Windows починаючи з версії 98.

Ресурси

Для нормальної роботи системи достатньо таких мінімальних засобів як:

Апаратні засоби комп'ютера:

- Процесор 1. 7 GHz
- Вінчестер 60 Gb
- Відео карта 256 Mb
- Оперативна пам'ять DDR2 512 Mb 400MHz

Програмні засоби:

- Операційна система Microsoft Windows XP SP2
- Microsoft NET Framework 2. 0

Перевірка вимог.

Відповідність вимог до мінімальних ресурсів системи

Таблиця 1

Характеристика	Міра
Продуктивність	Пошук рецепту повинен відбуватися мінімум як за 200мсек. Час на обробку інформації чеку не повинен перевищувати 1сек.
Розмір	Кількість записів обмежена дисковим простором.
Зручність для користувача	Час, потрібний для навчання не більший 10 хвилин. Середній час виконання будь-якої операції не більше 30 секунд.
Переносимість	Програма працює на всіх операційних системах Microsoft Windows починаючи з Windows98. Для перенесення без інсталяції достатньо перенести систему на новий комп'ютер.

Підтримка

Часто з'являються нові пристрої з якими система також повинна могли працювати. В такому випадку реалізація обміну і обробки даних між пристроєм та системою має бути в окремій бібліотеці. Це дасть змогу легко змінювати механізм обміну інформації з пристроєм при тому не зачіпаючи функцій самої системи.

2. 2. Розробка UML діаграми варіантів використання

Основна мета створення будь-якої програмної системи це створення програмного продукту, який допомагає користувачу виконувати свої повсякденні завдання. Для створення таких програм насамперед визначаються вимоги, яким повинна задовольняти система. Проте якщо дати користувачам написати ці вимоги на папері, то часто можна одержати список функцій, по якому важко судити чи буде майбутня система виконувати своє призначення і чи зможе вона полегшити користувачу виконання його роботи взагалі.

Для того, щоб точніше зрозуміти як повинна працювати система, все частіше використовується опис функціональності системи через варіанти

використання (UseCase, або прецеденти). Варіанти використання це – опис послідовності дій, які може здійснювати система у відповідь на зовнішні дії користувачів, або інших програмних систем. Варіанти використання відображають функціональність системи.

Діаграми варіантів використання описують функціональне призначення системи, або те, що система повинна робити. Мета розробки діаграм наступна:

- визначити загальні межі і предметну область;
- сформулювати загальні вимоги до функціональної поведінки проектованої системи;
- розробити початкову концептуальну модель системи її подальшої деталізації у формі логічних і фізичних моделей;
- підготувати початкову документацію для взаємодії розробників системи з замовниками і користувачами.

Діаграма варіантів використання – це граф спеціального вигляду, який є графічною нотацією для представлення конкретних варіантів використання, акторів, можливо деяких інтерфейсів, і відносин між цими елементами. При цьому окремі компоненти діаграми можуть бути поміщені в прямокутник, який позначає проектовану систему в цілому. Слід зазначити, що відносинами даного графа можуть бути тільки деякі фіксовані типи взаємозв'язків між акторами і варіантами використання, які в сукупності описують сервіси або функціональні вимоги до модельованої системи.

Суть діаграми варіантів використання полягає в наступному. Проектована система представляється у вигляді безлічі суті, або акторів, що взаємодіють з системою за допомогою варіантів використання. При цьому актором (actor), або дійовою особою називається будь-яка сутність, що взаємодіє з системою ззовні. Це може бути людина, технічний пристрій, програма, або будь-яка інша система, яка може служити джерелом дії на модельовану систему так, як визначить сам розробник. Варіант використання служить для опису сервісів, які система надає актору. Діаграма варіантів

використання може доповнюватися текстом пояснення, який розкриває сенс, або семантику складових її компонентів.

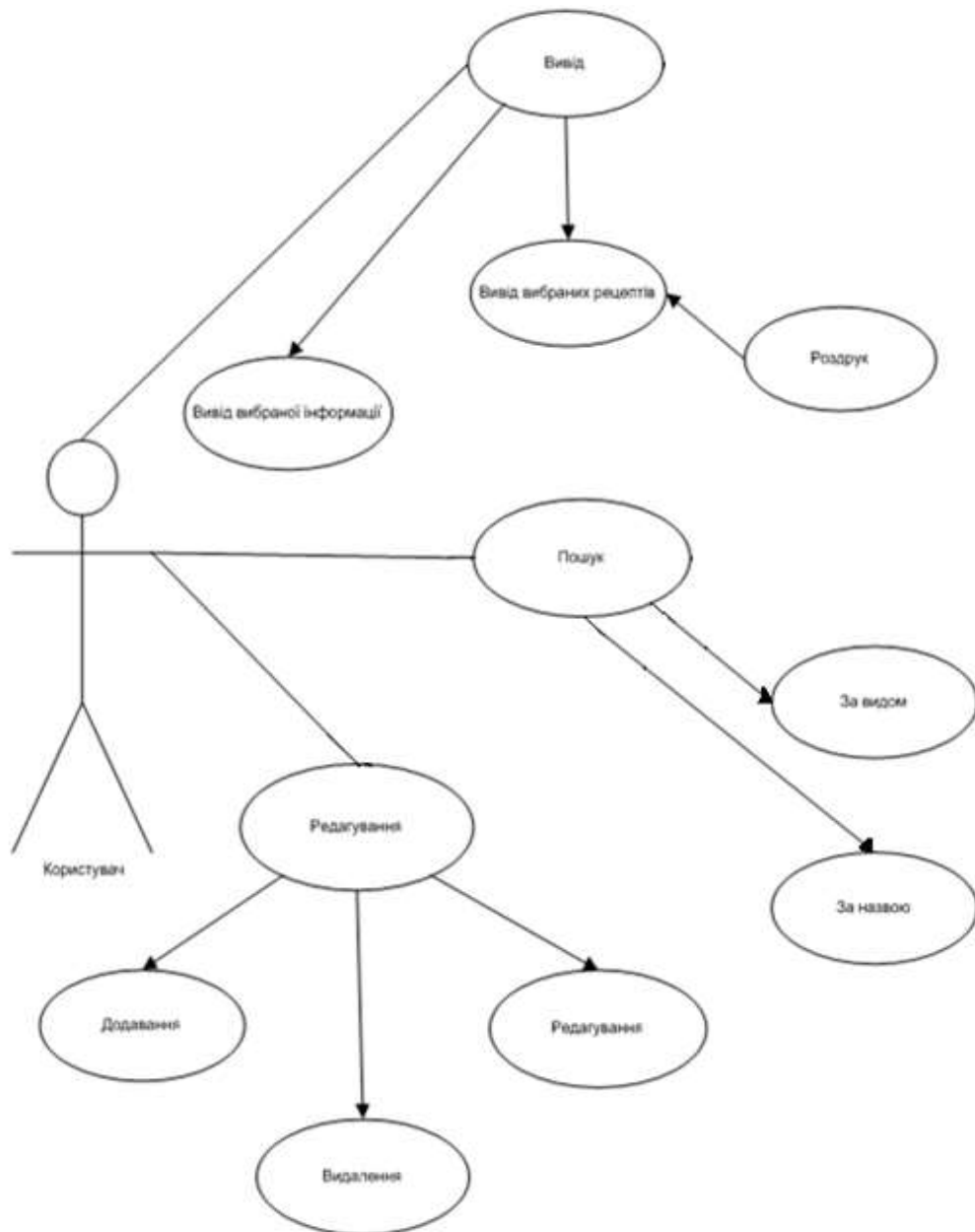


Рис. 2. Діаграма варіантів використання.

2. 3. Розробка UML діаграм поведінки системи

2. 3. 1. UML діаграма послідовності

Діаграма послідовності – в UML, діаграма послідовності відображає взаємодії об'єктів впорядкованих за часом. Зокрема, такі діаграми відображають задіяні об'єкти та послідовність відправлених повідомлень.

Діаграми послідовностей – це відмінний засіб документування поведінки системи, деталізації логіки сценаріїв використання, але є ще один спосіб – використовувати діаграми взаємодії. Діаграма взаємодії показує потік повідомлень між об'єктами системи і основні асоціації між ними і по суті, як вже було сказано вище, є альтернативою діаграмі послідовностей. Слід зазначити, що використання діаграм послідовностей або діаграм взаємодії – особистий вибір кожного проектувальника і залежить від індивідуального стилю проектування. На позначеннях, що застосовуються на діаграмі взаємодії, думаємо, не варто зупинятися детально. Тут все стандартно: об'єкти позначаються прямокутниками з підкресленими іменами (щоб відрізнити їх від класів), асоціації між об'єктами вказуються у вигляді з'єднувальних ліній, над ними може бути зображена стрілка із зазначенням назви повідомлення та його порядкового номера. Необхідність номеру повідомлення пояснюється дуже просто – на відміну від діаграм послідовностей, час на діаграмі взаємодії не показується у вигляді окремого вимірювання.

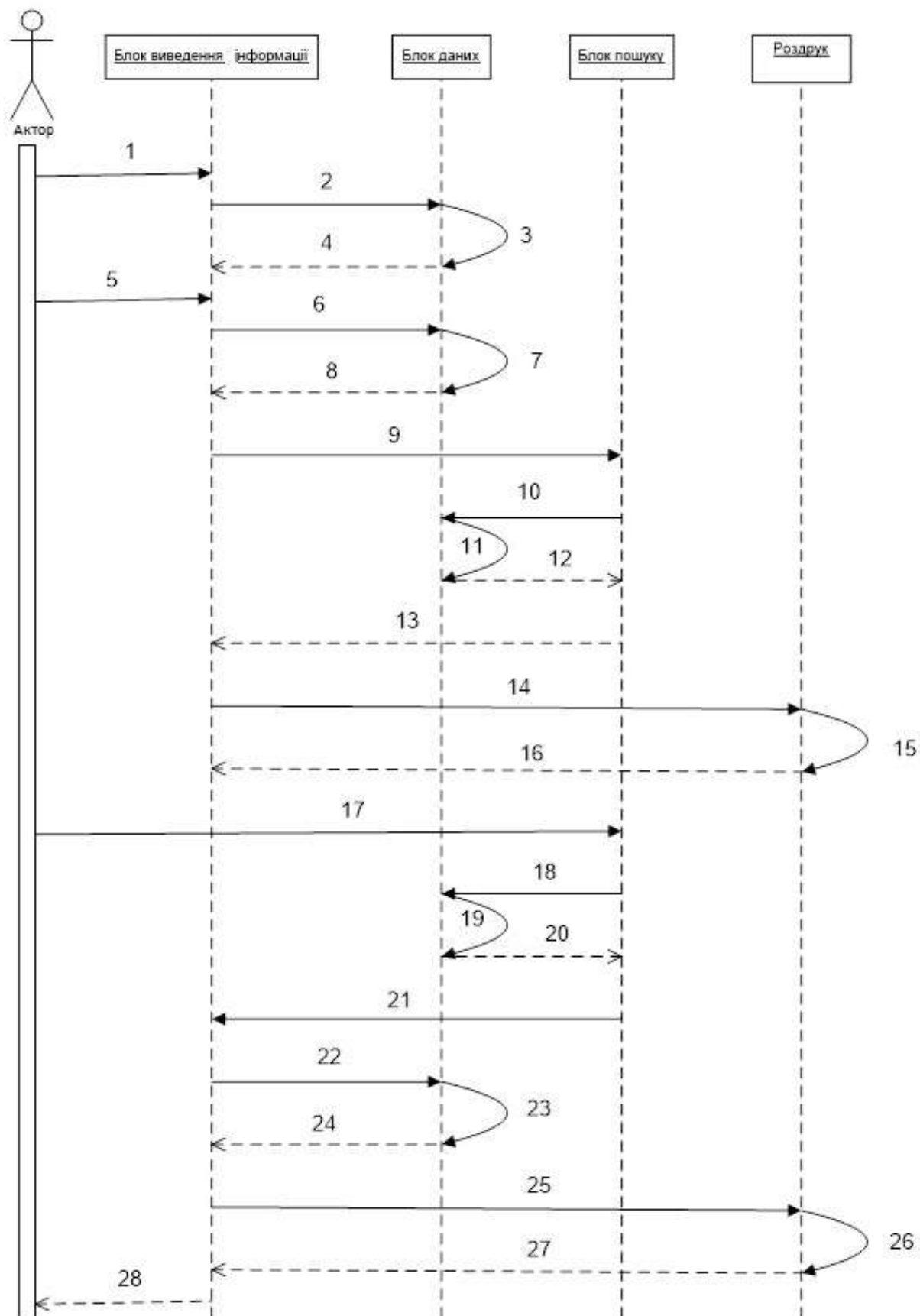


Рис. 3. Діаграма послідовностей.

Пояснення до рисунка:

1. Запит на виведення всіх страв з книги рецептів.
2. Отримання даних про всі рецепти.
3. Обробка даних (отримання даних з БД або з файлів).
4. Отримання результату.
5. Запит на формування змісту книги рецептів.
6. Отримання назв рецептів.
7. Обробка даних (отримання переліку рецептів з книги рецептів).
8. Отримання результату.
9. Запит на пошук та виведення інформації про вибраний рецепт.
10. Отримання даних про рецепт з БД.
11. Обробка даних (отримання даних про певний рецепт).
12. Отримання результатів.
13. Передача результату (результату пошуку).
14. Виведення інформації на роздрук.
15. Обробка даних.
16. Підтвердження роздруку.
17. Запит на пошук даних за певним критерієм.
18. Запит на отримання даних згідно з критеріями пошуку.
19. Обробка даних.
20. Отримання результатів.
21. Виведення списку назв рецептів що відповідають заданому критерію пошуку.
22. Запит на отримання даних про вибраний користувачем рецепт.
23. Обробка даних.
24. Отримання результату.
25. Виведення інформації на роздрук.
26. Обробка даних.
27. Підтвердження роздруку.
28. Вихід з програми.

2. 3. 2. UML діаграма діяльності

Діаграма діяльності – в UML, візуальне представлення графу діяльностей. Граф діяльностей є різновидом графу станів скінченного автомату, вершинами якого є певні дії, а переходи відбуваються по завершенню дій.

Дія є фундаментальною одиницею визначення поведінки в специфікації. Дія отримує множину вхідних сигналів, та перетворює їх на множину вихідних сигналів. Одна із цих множин, або обидві водночас, можуть бути порожніми. Виконання дії відповідає виконанню окремої дії. Подібно до цього, виконання діяльності є виконанням окремої діяльності, буквально, включно із виконанням тих дій, що містяться в діяльності. Кожна дія в діяльності може виконуватись один, два, або більше разів під час одного виконання діяльності. Щонайменше, дії мають отримувати дані, перетворювати їх та тестувати, деякі дії можуть вимагати певної послідовності. Специфікація діяльності (на вищих рівнях сумісності) може дозволяти виконання декількох (логічних) потоків, та існування механізмів синхронізації для гарантування виконання дій у правильному порядку.

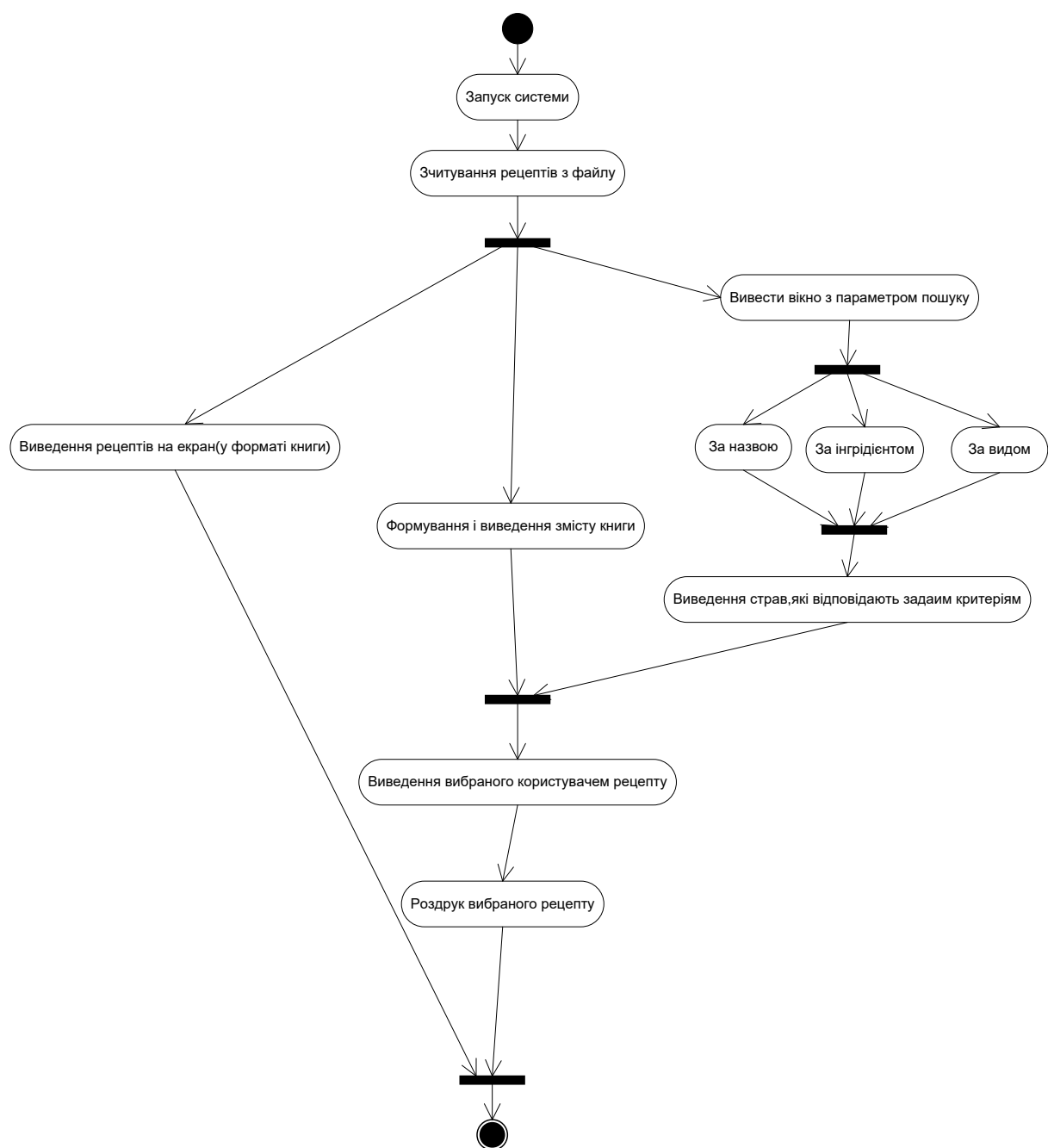


Рис. 4. Діаграма діяльності.

2. 4. Розробка графічного інтерфейсу програмних засобів комп'ютерної системи

Сучасність досить багата на досягнення в області програмного забезпечення та проектування користувацького інтерфейсу. З новими технологіями проектування інтерфейсу змінилось, але, як і раніше, базується на людському сприйнятті і пізнанні. Проектування користувацького

інтерфейсу – це більше, ніж просто розміщення на екрані керуючих елементів програми. З роками користувачі перейшли від інтерфейсів командного рядка до графічних інтерфейсів і зараз відкривають можливості об'єктно-орієнтованого користувацького інтерфейсу. Ідея полягає в тому, що при проектуванні інтерфейсу завжди потрібно думати про перспективи і не забувати, що програмне забезпечення створюється з розрахунку на потреби користувача, а не проектувальника. Проектування і побудова вдалого і практичного користувацького інтерфейсу – це й наука, й мистецтво.



Рис. 5. Пояснення дизайну.

На рис. 6 показано як виглядають рецепти у вигляді книги.



Рис. 6. Головна сторінка.

На рис. 7 показано як виглядають рецепти у вигляді списку.



Рис. 7. Вікно зі змістом.

РОЗДІЛ 3. Розробка програмного забезпечення системи

3. 1. Розробка UML діаграм класів

Діаграма класів – статичне представлення структури моделі. Відображає статичні (декларативні) елементи, такі як класи, типи, їх зміст та відношення. Діаграма класів, також, може містити позначення для пакетів та може містити позначення для вкладених пакетів. Також, діаграма класів може містити позначення деяких елементів поведінки, однак їх динаміка розкривається в інших типах діаграм.

Діаграма класів (classdiagram) служить для представлення статичної структури моделі системи в термінології класів об'єктно-орієнтованого програмування. На цій діаграмі показують класи, інтерфейси, об'єкти й кооперації, а також їхні відносини.

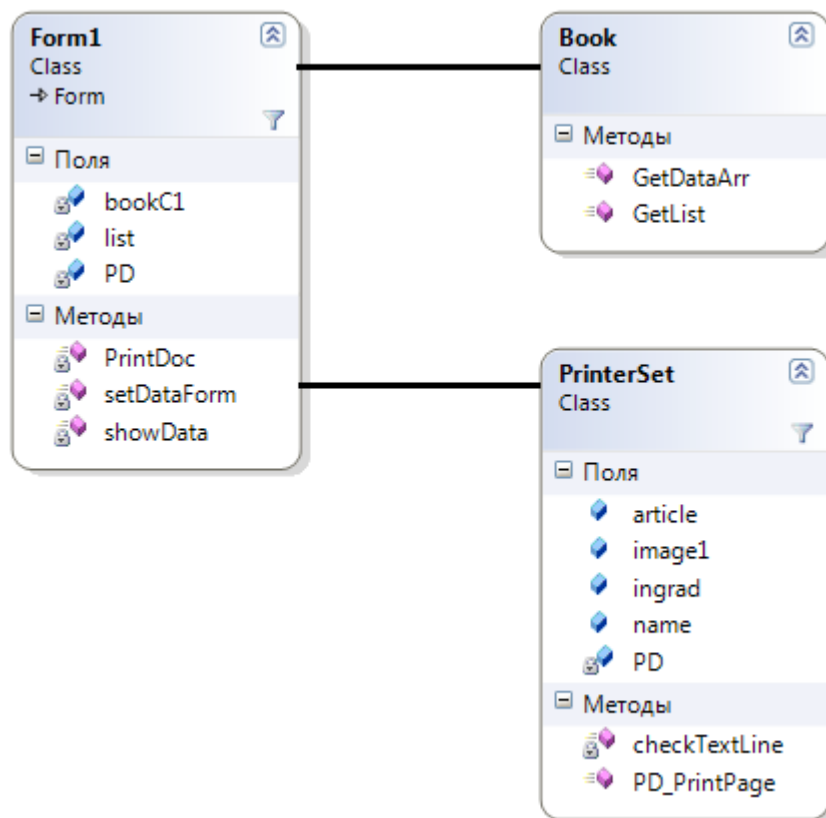


Рис. 8. Діаграма класів

3. 2. Опис класів програмного комплексу

Структура класу Form1.

Клас Form1 містить такі атрибути:

bookC1- ініціалізує новий екземпляр класу Book

list- створює двовимірний масив з усіма даними про рецепт

PD- ініціалізує новий екземпляр класу PrintDocument

Клас Form1 містить такі методи:

PrintDoc- друк

setDataForm- виведення інформації про рецепт

showData- виведення рецептів у форматі книжки

Структура класу PrinterSet.

Клас PrinterSet містить такі атрибути:

article- інструкція приготування

image1- візуальне зображення рецепту

ingrad- інгредієнти рецепту

name- назва рецепту

PD- друк

Клас PrinterSet містить такі методи:

checkTextLine- перевірка рядку

PD_PrintPage- друк сторінки

Структура класу Book.

Клас Book містить такі методи:

GetDataArr- отримання інформації у вигляді масиву

GetList- отримання списку

3. 3. Розробка структури бази даних

Схема баз даних (http://uk.wikipedia.org/wiki/Англійська_моваанг.databaseschema) – це структура системи баз даних описана формальною мовою, яка підтримується системою управління баз даних (СУБД) і відноситься до організації даних для створення плану побудови база даних з розподілом на таблиці. Формально схема баз даних являє собою набір формул (правил), які називаються обмеженнями цілісності. Обмеження цілісності забезпечують сумісність між всіма частинами схеми. Всі обмеження виражаються однією мовою.

Рецепти	
PK	ID рецепту
	Назва рецепту Картинка рецепту Інгредієнти рецепту Інструкція приготування рецепту

Рис. 9. Структура база даних.

РОЗДІЛ 4. Тестування програмного забезпечення

4. 1. Розробка тестів

Тестування програми електронна книга рецептів на основі функціонального тестування.

Резюме (Summary) 1: Перевірка функціонування кнопки завантаження інформації з БД.

Середовище виконання: Windows 7.

Опис (Description) – Перевіряємо чи при натисканні на відповідну кнопку виводяться рецепти у вигляді книжки.

Етапи для відтворення проблеми:

1. Відкриваю програму електронна книга рецептів.
2. Натискаю відповідну кнопку.

Очікуваний результат виконання (Expectedresult) : На екран повинна виводитись таблиця з рецептами у вигляді книжки.

Поточний результат виконання (Observedresult) : На екран вивелась таблиця з рецептами у вигляді книжки.

Складність (Severity) : рівень середній

Тест пройшов успішно.

Резюме (Summary) 2: Перевірка можливості редагування інформації про рецепт.

Середовище виконання: Windows 7.

Опис (Description) – Перевіряємо чи при натисканні на відповідну кнопку виведеться вікно в якому можливо редагувати інформацію про рецепт.

Етапи для відтворення проблеми:

1. Відкриваю програму електронна книга рецептів.
2. Натискаю відповідну кнопку.

Очікуваний результат виконання (Expectedresult) : Повинне виводитись вікно на екран в якому можна редагувати інформацію про рецепт.

Поточний результат виконання (Observedresult) : При натисканні на кнопку нічого не відбулось.

Складність (Severity) : рівень середній

Першочерговість (Priority) :

Р. 2. – виправити до кінця закінчення етапу тестування.

РОЗДІЛ 5. Розробка документів на супроводження програмного забезпечення

5. 1. Інструкція користувачеві

Щоб запустити програму «Електронна книга рецептів» необхідно відкрити файл запуску.

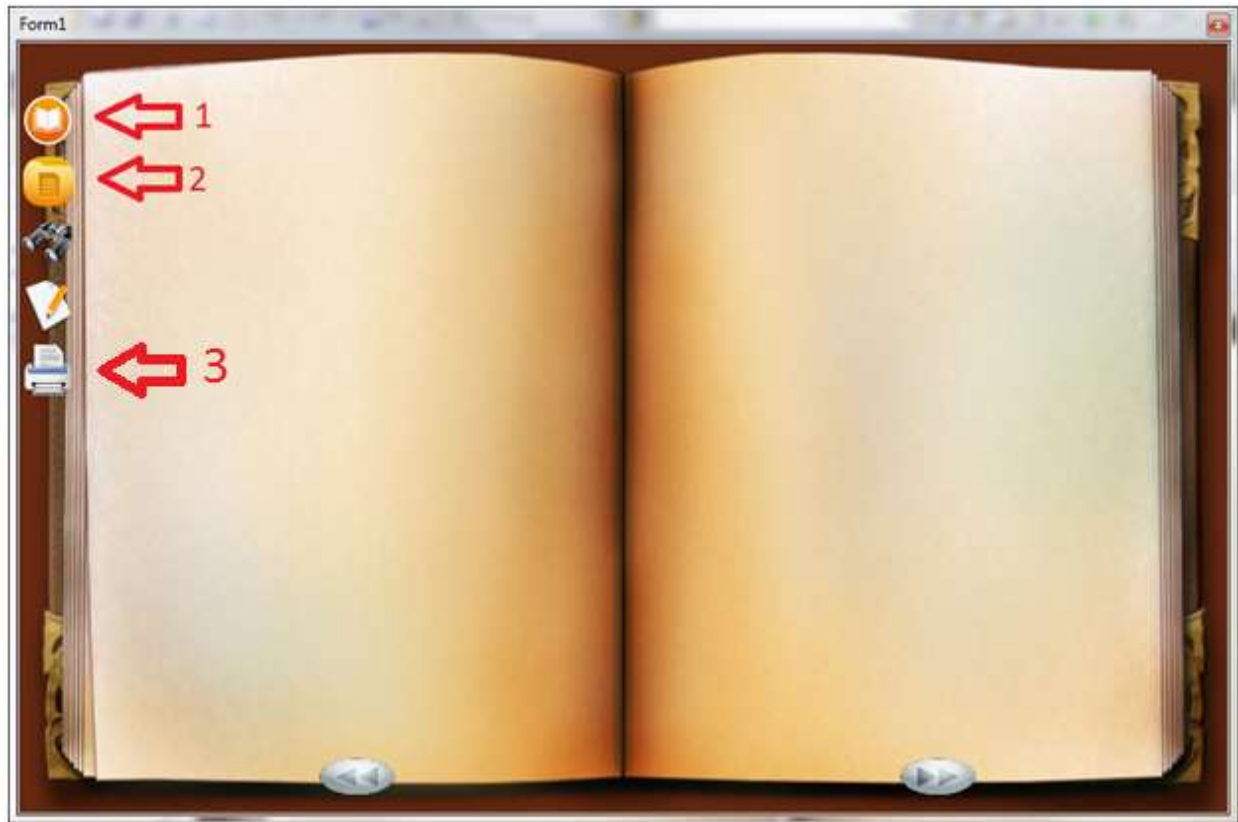


Рис. 10. Головне вікно програми.

Після чого необхідно завантажити дані про рецепти з БД за допомогою відповідної кнопки, після чого можна працювати з програмою. Після натискання на кнопку 1-рецепти виведуться на екран у вигляді книжки, як показано на рис. 11, якщо натиснути на кнопку 2- то рецепти виведуться у вигляді списку, як показано на рис. 12. Також можна роздрукувати вибраний рецепт, для цього необхідно натиснути на кнопку 3, після чого виведеться вікно яке зображено на рис. 13, на якому потрібно буде натиснути на кнопку «Друк».



Рис. 11. Виведення рецептів у вигляді книги.

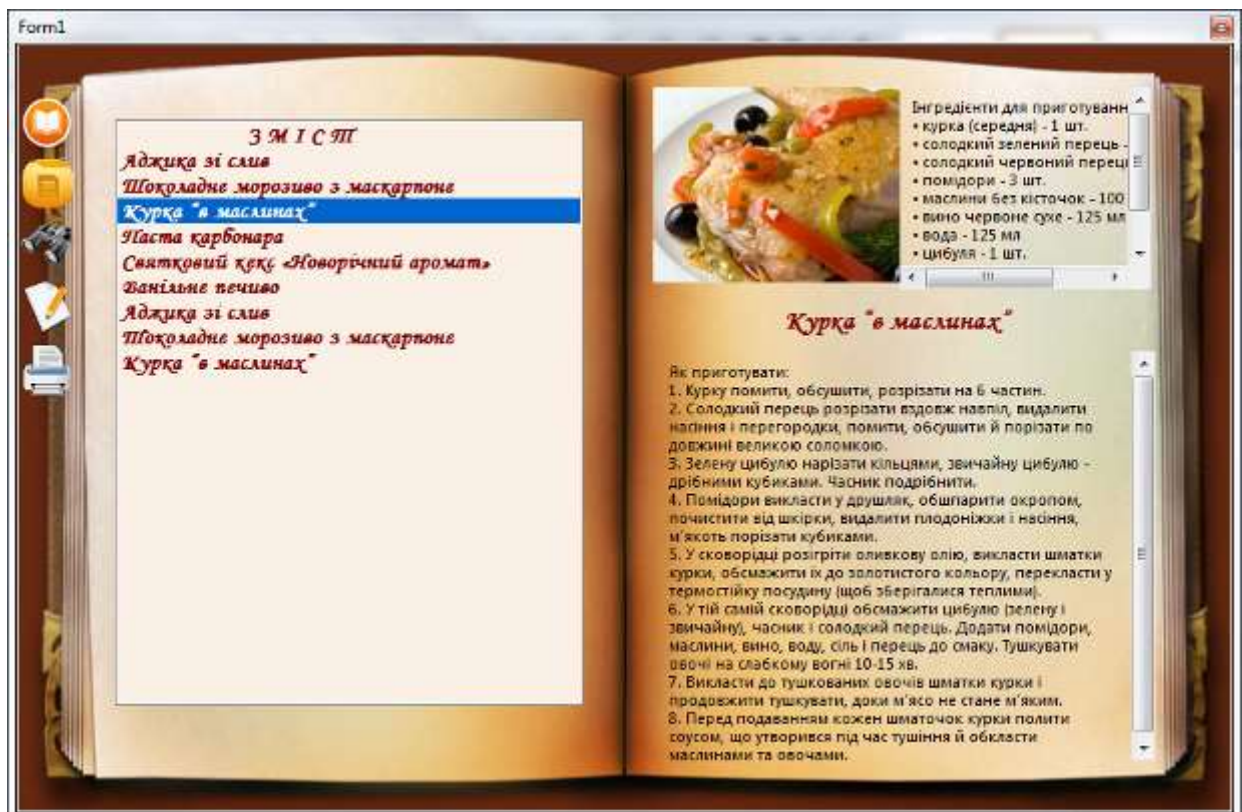


Рис. 12. Вікно зі змістом.

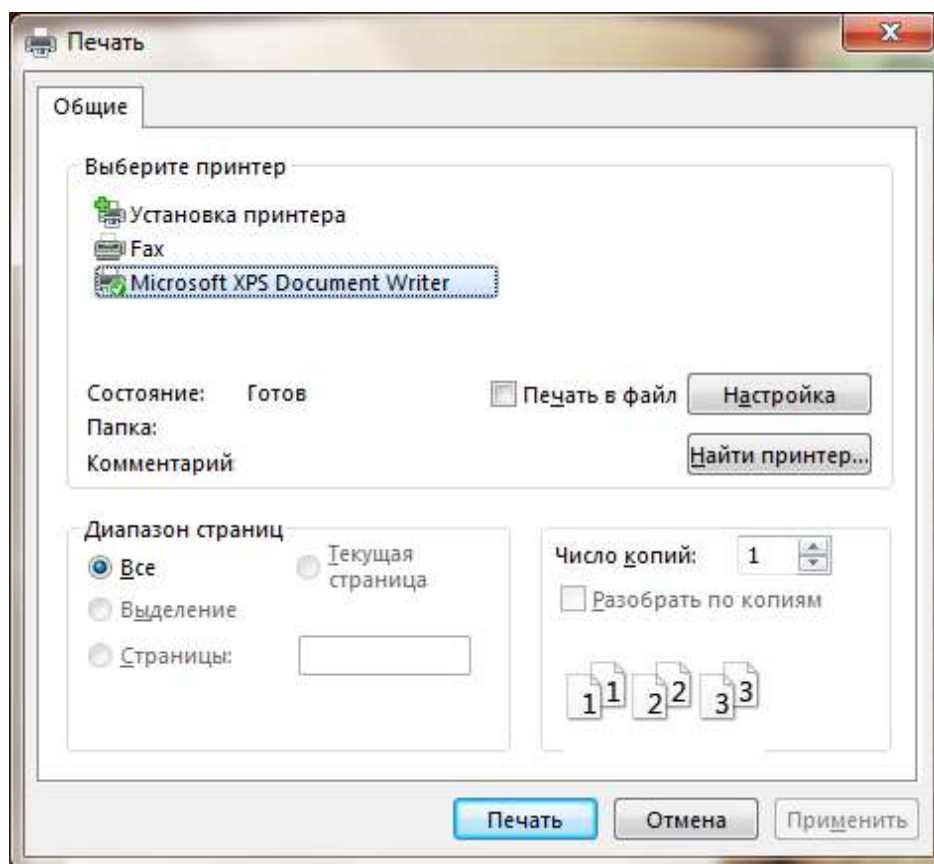


Рис. 13. Вікно друку.

На рис. 13 показано можливість роздруку вибраного рецепту.

Висновок

В ході виконання розрахункової роботи мовою C++ в середовищі Visual Studio 2019 реалізовано програму «Електронна книга рецептів». Також продемонстровано на прикладі можливості технології Windows Form на базі мови програмування C++.

Реалізовано такі функціональні вимоги:

- Здійснено запис та зчитування даних у файл (з файлу).
- Можливість виведення всіх рецептів на екран.
- Можливість виведення детальної інформації про вибраний рецепт.
- Можливість пошуку за назвою.
- Дані про рецепти зберігаються у відповідних текстових файлах.
- Можливість задання під кожен рецепт відповідного зображення.
- Можливість роздруку рецептів.
- Технологія Windows Forms дає можливість створювати зручний інтерфейс користувача.

Об'єктно-орієнтоване програмування дозволяє створювати програму, як набір користувацьких типів даних (класів), приховувати деталі реалізації, використовувати повторний код, інтерпретувати виклики процедур та функцій на етапі виконання (втілюючи основи ООП – інкапсулювання, поліморфізм, успадкування).

На закінчення терміну на виконання розрахункової роботи залишилися не реалізованими такі функціональні вимоги: можливість зміни даних (додавання, вилучення, редагування).

Універсальність програми в тому, що кожний користувач має повний доступ до бази даних категорій рецептів і до рецептів, має можливість групувати рецепти таким чином, щоб йому було зручно ними користуватися.

Список використаної літератури

1. Гордон Хогенсон "C++/CLI: язык Visual C++ для среды .NET".
Издательство: Вильямс, 2007 С. 464.
2. Б. Пахомов C/C++ и MS Visual C++ 2008 для начинающих (+ DVD-ROM).
3. Фаулер М., Скотт К. UML. Основы. — Пер. с англ. — СПб: Символ-Плюс, 2002. — 192 с., ил. ISBN 5-93286-032-4
4. Буч Г., Рамбо Дж., Джекобсон А. Язык UML. Руководство пользователя. — Пер. с англ. — М.: ДМК, 2000. — 432 с.
5. Полный справочник по C++, 4-е издание, Герберт Шилдт.
6. Язык программирования C++ (C++11). Лекции и упражнения, 6-е издание, Стивен Прата
7. Visual C++ 2010: полный курс, Айвор Хортон.
8. Программирование: принципы и практика использования C++, исправленное издание, Бьярне Страуструп.

Додаток

Програмний код...

Орієнтований перелік тем для розрахункової роботи

1. Розробка програмного комплексу системи "Електронна енциклопедія" (*галузь знань (тематика) – на власний вибір*).
2. Розробка програмного комплексу системи "Електронний фотоальбом".
3. Розробка програмного комплексу системи "Електронна записна книжка".
4. Розробка програмного комплексу системи "Електронна книга рецептів".
5. Розробка програмного комплексу системи "Електронний телефонний довідник".
6. Розробка програмного комплексу системи "Тестування знання англійської мови".
7. Розробка програмного комплексу системи "Навчальна система для дітей" (*тематика – на власний вибір*).
8. Розробка програмного комплексу системи "Динамічна програма анімації".
9. Розробка програмного комплексу системи "Верстання статей до журналу".
10. Розробка системи "Довідник руху електричного транспорту м. Львова".
11. Розробка системи "Довідник руху маршрутного транспорту м. Львова".
12. Розробка графічного редактора .
13. Розробка системи автоматизації роботи диспетчера кас автовокзалу міста Львові.
14. Розробка системи автоматизації роботи менеджера автомагазину.
15. Розробка системи автоматизації роботи продавця-консультанта магазину парфумів.
16. Розробка системи автоматизації роботи продавця-консультанта магазину спортивних товарів.
17. Розробка системи автоматизації роботи продавця-консультанта аптеки.
18. Розробка системи автоматизації роботи менеджера спортивного клубу.
19. Розробка системи автоматизації роботи продавця-консультанта піцерії.

20. Розробка системи автоматизації обробки результатів сесії студентів і нарахування стипендії.
21. Розробка системи автоматизації роботи диспетчера фірми, яка займається експортом промислових товарів.
22. Розробка системи автоматизації роботи диспетчера торгової фірми "Євробудсвіт" (*фірма на власний вибір*) по отриманню та виконанню замовлень від клієнта.
23. Розробка системи автоматизації роботи диспетчера залізничного вокзалу.
24. Розробка програмного забезпечення КС автоматизації роботи продавця автомагазину.
25. Розробка програмного забезпечення КСУ доступом до інформаційного ресурсу з розподілом прав користувача.
26. Розробка комплексу програм для розважальної гри «Minesweeper (Сапер)» .
27. Розробка комплексу програм для розважальної гри «Судоку».
28. Розробка програмного забезпечення КС моделювання структур графів.
29. Розробка системи автоматизації роботи диспетчера маршрутного таксі №...
30. Розробка системи автоматизації роботи диспетчера таксопарку.
31. Розробка системи автоматизації обліку кадрів підприємства.
32. Розробка програмного забезпечення для автоматизованої системи «Деканат».
33. Розробка програмного забезпечення автоматизованої системи ведення власної бібліотеки.
34. Розробка програмного забезпечення автоматизованої системи ведення власної аудіотеки.
35. Розробка програмного забезпечення автоматизованої системи ведення власної відеотеки.
36. Розробка системи автоматизації роботи диспетчера туристичної фірми.

37. Розробка системи автоматизації роботи диспетчера салону прокату відеофільмів.
38. Розробка програмного забезпечення комп'ютерної системи контролю відвідування занять студентами.
39. Розробка інтерфейсу програми для управління базою даних медикаментів в аптеці .
40. **Тема на власний вибір за попереднім погодженням з викладачем.**